Apex Institute of Technology

Computer Science & Engineering

Experiment 6

Name: Kaushik Kaundal **UID: 24BAI70562**

Branch: B.E. CSE (AIML) **Section: 24AIT_KRG-G1**

Semester: 4 **Date of Performance: 13.03.2026**

Subject Name: Database Management System **Subject Code: 24CSH-298**

AIM: To understand the concept and working of **cursors** in PL/SQL for **row-by-row data processing**, and to analyze how **implicit cursors**, **explicit cursors**, and **cursor attributes** are used to implement business logic on multiple rows in a database table.

OBJECTIVES:

- To implement and analyze the use of **implicit cursors**, **explicit cursors**, and **cursor attributes** for processing multiple rows from a database table and applying business logic effectively.

SOFTWARE REQUIREMENTS:

- Database Management System:
  - PostgreSQL Database
- Database Administration Tool / Client Tool:
  - pgAdmin

PRACTICAL/EXPERIMENT STEPS:

1. An employees table was created in the Oracle database with columns such as emp_id, emp_name, and emp_sal to store employee details.
2. Sample employee records were inserted into the table to provide data for testing cursor operations.
3. The concept of implicit cursors was studied to understand how Oracle automatically handles DML operations like UPDATE.
4. A PL/SQL block using an implicit cursor was written to update employee salary and verify execution using SQL%FOUND and SQL%ROWCOUNT.

5. The concept of explicit cursors was explored to process multiple rows returned by a SELECT query.
6. A PL/SQL program using an explicit cursor was written to fetch employee records one by one and apply business logic.
7. The programs were executed in Oracle FreeSQL, and the output results were verified for correctness.

PROCEDURE:

1. The Oracle FreeSQL environment was opened to access the database.
2. A new employees table was created with fields such as emp_id, emp_name, and emp_sal.

```
Table EMPLOYEES created.

Elapsed: 00:00:00.024
```

3. Sample employee records were inserted into the table using INSERT statements.

	EMP_ID	EMP_NAME	EMP_SAL
1	101	Amit	30000
2	102	Riya	45000
3	103	Karan	25000
4	104	Neha	50000
5	105	Arjun	35000

4. A PL/SQL block using an implicit cursor was written to update employee salary and check execution status using SQL%FOUND and SQL%ROWCOUNT.

```
Employee salary updated successfully
Rows affected: 1


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.015
```

5. An explicit cursor was declared to retrieve multiple employee records from the table.
6. The cursor was opened, and records were fetched one by one using a LOOP structure, and business logic was applied.

```
ID: 101 Name: Amit Salary: 32000
Status: Normal Salary
ID: 102 Name: Riya Salary: 45000
Status: High Salary
ID: 103 Name: Karan Salary: 25000
Status: Normal Salary
ID: 104 Name: Neha Salary: 50000
Status: High Salary
ID: 105 Name: Arjun Salary: 35000
Status: Normal Salary


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.092
```

7. The cursor was closed after processing all records, and the output was observed and recorded.

CODE:

```
CREATE TABLE employees (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(50),
    emp_sal NUMBER
);

INSERT INTO employees VALUES (101, 'Amit', 30000);
INSERT INTO employees VALUES (102, 'Riya', 45000);
INSERT INTO employees VALUES (103, 'Karan', 25000);
INSERT INTO employees VALUES (104, 'Neha', 50000);
INSERT INTO employees VALUES (105, 'Arjun', 35000);

COMMIT;

DECLARE
    v_emp_id employees.emp_id%TYPE := 101;
    v_new_sal employees.emp_sal%TYPE := 32000;
BEGIN
    UPDATE employees
    SET emp_sal = v_new_sal
    WHERE emp_id = v_emp_id;

    IF SQL%FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee salary updated successfully');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Employee not found');
    END IF;

    DBMS_OUTPUT.PUT_LINE('Rows affected: ' || SQL%ROWCOUNT);
END;
/
DECLARE
```

```
    CURSOR emp_cursor IS
        SELECT emp_id, emp_name, emp_sal FROM employees;

    v_id employees.emp_id%TYPE;
    v_name employees.emp_name%TYPE;
    v_sal employees.emp_sal%TYPE;
BEGIN
    OPEN emp_cursor;

    LOOP
        FETCH emp_cursor INTO v_id, v_name, v_sal;
        EXIT WHEN emp_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE('ID: ' || v_id ||
                             ' Name: ' || v_name ||
                             ' Salary: ' || v_sal);

        IF v_sal > 40000 THEN
            DBMS_OUTPUT.PUT_LINE('Status: High Salary');
        ELSE
            DBMS_OUTPUT.PUT_LINE('Status: Normal Salary');
        END IF;

    END LOOP;

    CLOSE emp_cursor;
END;
/
```

I/O ANALYSIS:

1. Implicit Cursor Output
   Displays messages indicating whether the employee salary was successfully updated and shows the number of rows affected using SQL%FOUND and SQL%ROWCOUNT.

```
Employee salary updated successfully
Rows affected: 1


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.015
```

2. Explicit Cursor Output
   Displays employee details (ID, Name, Salary) for each record fetched from the table.

```
ID: 101 Name: Amit Salary: 32000
Status: Normal Salary
ID: 102 Name: Riya Salary: 45000
Status: High Salary
ID: 103 Name: Karan Salary: 25000
Status: Normal Salary
ID: 104 Name: Neha Salary: 50000
Status: High Salary
ID: 105 Name: Arjun Salary: 35000
Status: Normal Salary


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.092
```

3. Business Logic Result
   Shows classification of employees based on salary (e.g., "High Salary" or "Normal Salary") for each row processed.

LEARNING OUTCOMES:

1. Understood the role of cursors in PL/SQL for handling multi-row query results.
2. Differentiated between implicit cursors and explicit cursors.
3. Used cursor attributes such as `%FOUND`, `%NOTFOUND`, `%ROWCOUNT`.
4. Developed PL/SQL programs that process database records row by row.
5. Applied cursor-based logic to real-world business scenarios.